

Telecommunication Assignment -4



Faculty of Engineering

Department of Electrical Engineering and Computer Science

Hochschule Wismar University of Applied Sciences

Technology, Business and Design

Telecommunication Project 2023

Guided by : Prof. Dr.-Ing Toralf Renkwitz

Date : 17-05-2023

Anusree Kundu	(460596)
Chandrayudula Ramachandra Srinivasa Prasad Tharun Kumar	(433303)
Lohith Reddy Prane	(431031)
Neha Manjunath	(451772)
Shreyas Ravi Vasista	(457183)

LIST OF CONTENTS

Serial No	Title	Page No
1	Task - 1	1
2	Task - 2	5
3	References	10

LIST OF FIGURES

Fig No	Title	Page No
1.1	Complex Signal with real and imaginary part	4
1.2	4-State QAM	4
1.3	Time Series Plots at 1, 25 and 100 CI	4
1.4	SNR v/s CI Plot	4
2.1	Gaussian Distribution Plot with and without noise	9
2.2	Gaussian Spectra Plot	9
2.3	Spectral Plot for 1 NCI	9
2.4	Spectral Plot for 25 NCI	9
2.5	Spectral Plot for 100 NCI	9
2.6	SNR v/s NCI Plot	9

Task - 1

Create a long complex valued time series signal, perhaps a 2 or 4 state QAM for 400 samples and add complex noise to the time series

- Derive a "signal to noise ratio"
- Proposal: derive the deviation for each sample to the nominal QAM state position
- apply coherent integrations: 1, 5, 10, 25, 50, 100
- plot the time series for 1, 25 and 100 CI
- plot "SNR" over CI, any mathematical approximation visible?

Coherent integration : It is a technique that preserves the phase information of the received signal and adds it up in the complex domain. This means that the amplitude and the angle of the signal are both considered in the integration process. Coherent integration can enhance the SNR by a factor of N , where N is the number of pulses or samples integrated. However, coherent integration requires a stable reference signal and a high synchronization accuracy between the transmitter and the receiver. Moreover, coherent integration is sensitive to target motion and Doppler shifts, which can cause phase errors and degrade the performance.

Script :

```
import numpy as np
import matplotlib.pyplot as plt

# define QAM constellation
constellation = np.array([1+1j, -1+1j, -1-1j, 1-1j])

npts=400
t = np.linspace(-2*np.pi, 2*np.pi, npts, endpoint=True)
x = t
iq=np.zeros(1)
for jj in range(npts):
    iq=np.append(iq,np.tile(np.round(np.random.randn(1))+1j*np.round(np.random.randn(1))),npts))

iq=iq[1:]

x=np.arange(1,len(iq)+1)
```

```

plt.figure()
plt.subplot(211)
plt.plot(x,np.real(iq))
plt.ylabel('Re')
plt.xlim([0,20000])
plt.subplot(212)
plt.plot(x,np.imag(iq))
plt.ylabel('Im')
plt.xlim([0,20000])

# generate long time series with persistent states
num_samples = 10000
state_duration = 400
num_states = int(num_samples / state_duration)
states = np.random.randint(0, 4, size=num_states)
signal = np.repeat(constellation[states], state_duration)
plt.figure()
plt.title('4 - State QAM')
plt.plot(np.real(constellation),np.imag(constellation),'.')
plt.grid('on')

# add complex noise
noise_power = 0.01
noise=np.sqrt(noise_power/2)*(np.random.randn(num_samples) + 1j*np.random.randn(num_samples))
signal_noisy = signal + noise

# derive signal to noise ratio
signal_power = np.mean(np.abs(signal)**2)
noise_power_est = np.mean(np.abs(noise)**2)
snr = 10 * np.log10(signal_power / noise_power_est)

# apply coherent integrations
cis = [1, 5, 10, 25, 50, 100]
integrated_signals = []
for ci in cis:
    num_integrations = int(num_samples / ci)

```

```

integrated_signal = np.zeros(num_integrations, dtype=complex)
for i in range(num_integrations):
    start_idx = i*ci
    end_idx = start_idx + ci
    integrated_signal[i] = np.sum(signal_noisy[start_idx:end_idx])
integrated_signals.append(integrated_signal)

# plot time series for selected coherent integrations
fig, axs = plt.subplots(3, 1, figsize=(10, 10), sharex=True)
axs[0].plot(signal_noisy[:state_duration*5].real, label='noisy signal')
axs[0].plot(signal[:state_duration*5].real, label='original signal')
axs[0].legend()
axs[0].set_title('Time series for 1 CI')
axs[1].plot(signal_noisy[:state_duration*25:5].real, label='noisy signal')
axs[1].plot(signal[:state_duration*25:5].real, label='original signal')
axs[1].legend()
axs[1].set_title('Time series for 25 CI')
axs[2].plot(signal_noisy[:state_duration*100:20].real, label='noisy signal')
axs[2].plot(signal[:state_duration*100:20].real, label='original signal')
axs[2].set_xlim([0, 700])
axs[2].legend()
axs[2].set_title('Time series for 100 CI')
plt.show()

# calculate SNR for different number of coherent integrations
snrs = []
for integrated_signal in integrated_signals:
    signal_power_int = np.mean(np.abs(integrated_signal)**2)
    noise_power_est_int = noise_power_est * ci
    snr_int = 10 * np.log10(signal_power_int / noise_power_est_int)
    snrs.append(snr_int)

# plot SNR vs CI
plt.plot(cis, snrs)
plt.xlabel('Coherent Integrations (CI)')
plt.ylabel('Signal to Noise Ratio (SNR) [dB]')

```

```
plt.title('SNR vs Coherent Integrations')
```

```
plt.show()
```

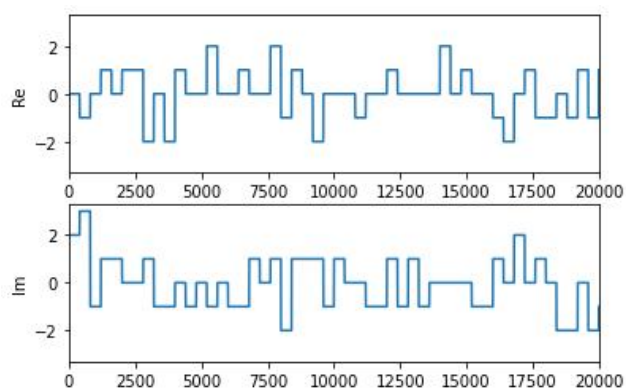


Fig 1.1 : Complex Signal with real and imaginary part

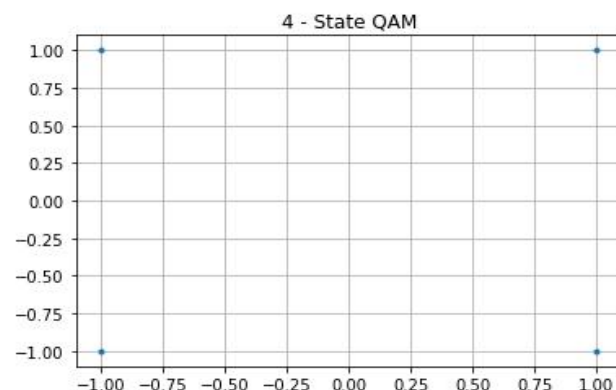


Fig 1.2 : 4-State QAM

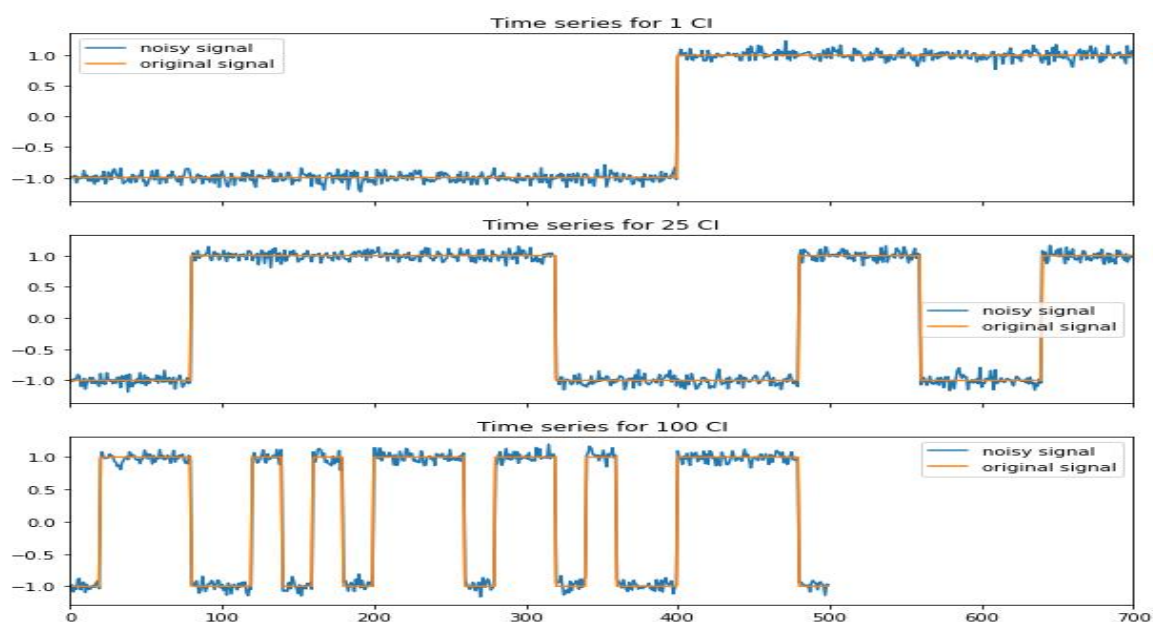


Fig 1.3 : Time Series Plots at 1, 25 and 100 CI

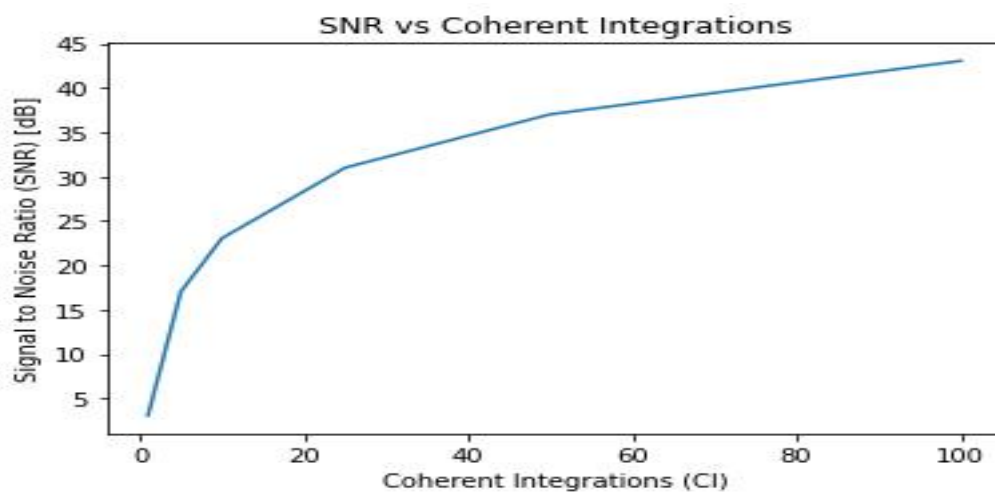


Fig 1.4 : SNR v/s CI Plot

Task - 2

Repeat the equivalent for incoherent integrations. Create a gaussian-like signal in the spectral domain and add noise

- apply 1, 5, 10, 25, 50, 100 incoherent integration
- do not split just one time series
- derive a "signal to noise ratio"
- plot the integrated spectrum for 1, 25 and 100 NCI
- investigate and plot the dependence of "SNR" to the number of integrated spectra

Incoherent Integration : It is a technique that discards the phase information of the received signal and adds up only the magnitude or power of the signal. This means that only the amplitude of the signal is considered in the integration process. Non-coherent integration can enhance the SNR by a factor of $\sqrt{N}$, where N is the number of pulses or samples integrated. However, non-coherent integration has a lower SNR gain than coherent integration and can lose some target information that is encoded in the phase. Moreover, non-coherent integration can introduce some noise due to the squaring operation, which can affect the detection threshold.

Script :

```
import numpy as np
import matplotlib.pyplot as plt

npts=1000
t = np.linspace(-2*np.pi, 2*np.pi, npts, endpoint=True)
x=t
sigma2 = 20
sigma = np.sqrt(sigma2)

y1 = 1 / (1*np.pi*sigma) * np.exp(-(x)**2 / (2*sigma2))
n = np.random.randn(npts)/10
y1n = y1 + n

plt.figure()
```

```

plt.subplot(211)
plt.plot(x,y1)
plt.title('Gaussian Distribution')
plt.grid('on')
plt.ylabel('Amplitude')
plt.xlabel('Time[Sec]')
plt.subplot(212)
plt.plot(x,y1)
plt.plot(x,y1n)
plt.title('Gaussian Distribution with noise')
plt.grid('on')
plt.ylabel('Amplitude')
plt.xlabel('Time[Sec]')
plt.legend(('Signal', 'Signal+Noise'))

# Perform FFT
def make_fft_DS(x,t):

    ts = (t[-1]-t[0]) / (len(t)-1)
    Fs=1/ts
    L = len(x)
    spectr = np.fft.fft(x)/L;
    spectr= np.fft.fftshift(spectr)
    dF = Fs/L;

    if L%2==0:
        f=np.arange(-1*(Fs/2), Fs/2, dF)
        # f=f[0:-1];
    else:
        f=np.arange( -1*(Fs/2 -dF/2), (Fs/2 -dF/2),dF)

    return f, spectr

# Performs non-coherent integration
def make_icoh(t, y, icoh):
    nptsn=int(np.floor(len(y)/icoh))

```



```

Yi=np.empty(nptsn)
tn=np.empty(nptsn)
speci=np.zeros([nptsn,icoh])+1j*np.zeros([nptsn,icoh])

# One time series
for i in range(0,icoh):
    yn=y[i*nptsn:i*nptsn+nptsn]
    tn=t[i*nptsn:(i+1)*nptsn]
    fi,Yi=make_fft_DS(yn,tn)
    # Individual spectra
    speci[:,i]=Yi
    # Integrated spectrum
    spectri=np.mean(abs(speci), axis=1)/np.sqrt(icoh)
    if len(fi)>len(spectri):
        fi=fi[:-1]

return fi,spectri

# Derive FFT of noisy signal
f0,spectr0= make_fft_DS(y1,t)
# Derive FFT of noisy signal
f,spectr= make_fft_DS(y1n,t)

plt.figure()
plt.plot(f0,abs(spectr0))
plt.plot(f,abs(spectr))
plt.title('Gaussian Spectra')
plt.grid('on')
plt.ylabel('Amplitude')
plt.xlabel('Freq[Hz]')
plt.legend(('Signal', 'Signal+Noise'))

# Perform Non Coherent integration with different averaging
spectra = []
icoh =[1, 5, 10, 25, 50, 100]
for count in icoh:

```

```

fi,spectri = make_icoh(t, y1n, count)

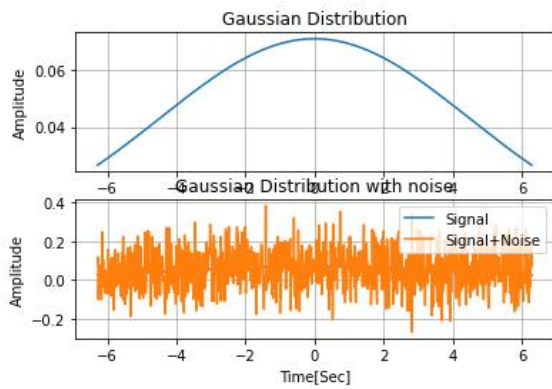
spectra.append(spectri)
plt.figure()
plt.plot(f,abs(spectr))
plt.plot(fi,abs(spectri),'*-')
plt.title(str(count)+' spectral (icoh) ')
plt.grid('on')
plt.ylabel('Amplitude')
plt.xlabel('Freq[Hz]')

# Plot the integrated spectrum for 1, 25 and 100 NCI
if count in [1, 25, 100]:
    plt.figure()
    plt.plot(fi, abs(spectri))
    plt.title(str(count) + ' spectral (NIC incoherent)')
    plt.grid('on')
    plt.ylabel('Amplitude')
    plt.xlabel('Freq[Hz]')

# Calculate signal to noise ratio
snr = []
spectr0 = np.fft.fft(y1n)
spectr0_abs = abs(spectr0)
snr.append(np.sum(spectr0_abs)/np.sum(np.abs(spectra[0])))
for i in range(len(icoh)-1):
    snr.append(np.sum(spectr0_abs)/np.sum(np.abs(spectra[i+1])))

# Plot signal to noise ratio
plt.figure()
plt.plot(icoh, snr)
plt.title('Signal to Noise Ratio vs Incoherent Integration')
plt.grid('on')
plt.ylabel('Signal to Noise Ratio')
plt.xlabel('Incoherent Integration')

```



**Fig 2.1 : Gaussian Distribution Plot
with and without noise**

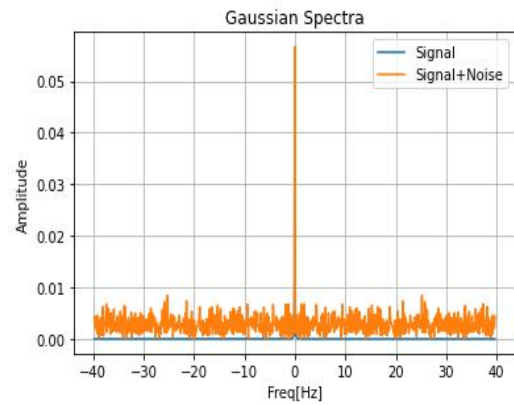


Fig 2.2 : Gaussian Spectra Plot

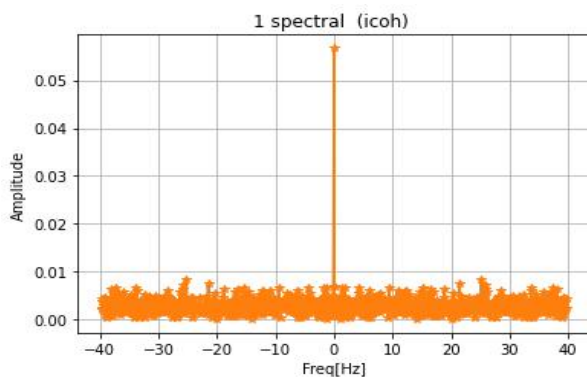


Fig 2.3 : Spectral Plot for 1 NCI

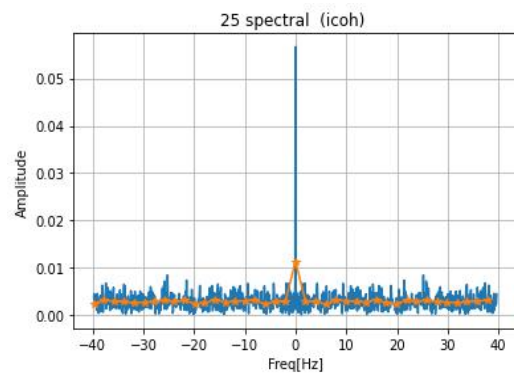


Fig 2.4 : Spectral Plot for 25 NCI

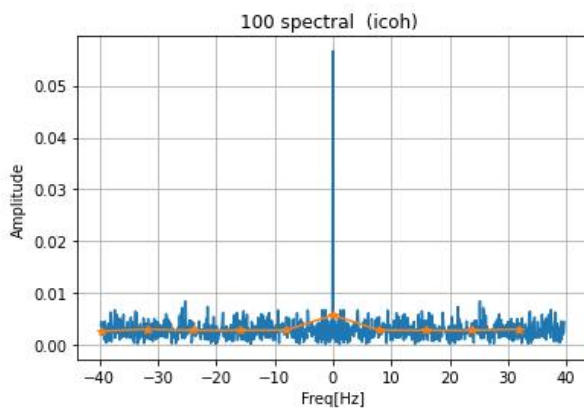


Fig 2.5 : Spectral Plot for 100 NCI

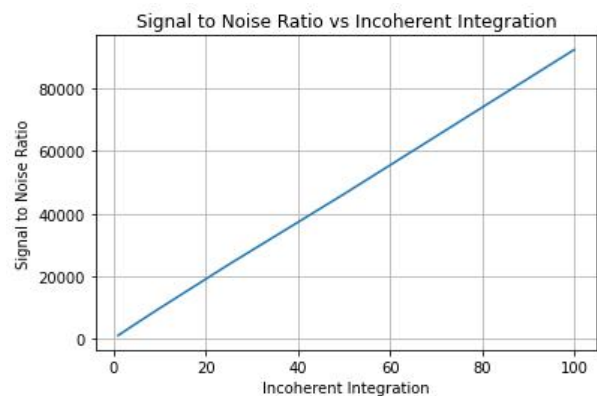


Fig 2.6 : SNR v/s NCI Plot

References

[1] 'What are the trade-off between coherent and non-coherent integration in radar detection ? ' [online]

<https://www.linkedin.com/advice/3/what-trade-offs-between-coherent-non-coherent-integration> [accessed 14.05.2023]

[2] 'Scipy.signal.coherence' [online]

<https://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.coherence.html>

[accessed 15.05.2023]